Roll No: _____ **Date:** _____

Experiment-7
Page Replacement Algorithms

Page replacement is a memory management technique used in operating systems to manage the contents of physical memory. When a process needs a page that is not currently in main memory, a page fault occurs, and the system must bring the required page from secondary storage (like a hard disk) into memory. If the memory is already full, one of the existing pages must be replaced; this decision is made by a page replacement algorithm. The goal is to minimize the number of page faults and improve overall system performance. Common algorithms include FIFO, LRU, and Optimal page replacement.

Key Terminology

- **Page:** A fixed-size block of logical memory (from a process) that is stored in the secondary memory (like a hard disk).
- **Frame:** A fixed-size block of physical memory (RAM) that can hold one page at a time.
- **Page Fault:** It occurs when a process tries to access a page that is not currently present in the main memory.
- **Page Hit:** It occurs when the required page is already available in the main memory.
- **Hit Ratio:** The ratio of the number of page hits to the total number of memory accesses.

$$\text{Hit Ratio} = \frac{\text{Number of Page Hits}}{\text{Total Memory Accesses}}$$

- **Miss Ratio:** The ratio of the number of page faults (misses) to the total number of memory accesses.

$$\text{Miss Ratio} = \frac{\text{Number of Page Faults}}{\text{Total Memory Accesses}}$$

Page Replacement Algorithms

1. FIFO (First-In First-Out) Page Replacement

- **Concept:** The page that entered memory first is replaced first when a new page needs to be loaded.
- **Example:** If the order of page loading is 1, 2, 3, 4 — when 5 arrives, page 1 (the oldest) is replaced.
- **Advantages:** Simple to understand and implement. Requires minimal overhead for tracking pages.

- **Disadvantages:** May remove frequently used pages (poor performance). Suffers from Belady's Anomaly (increasing frames may increase page faults).

2. LRU (Least Recently Used) Page Replacement

- **Concept:** The page that has not been used for the longest time is replaced first.
- **Example:** If pages used recently are 2, 3, 4, and 1 is least recently used, then page 1 is replaced.
- **Advantages:** Performs better than FIFO in most cases. Based on actual page usage patterns.
- **Disadvantages:** Requires hardware or software support to track usage. Higher implementation cost (time and space complexity).

3. Optimal Page Replacement

- **Concept:** The page that will not be used for the longest time in the future is replaced.
- **Example:** If future reference sequence is known, replace the page that will appear farthest ahead.
- **Advantages:** Produces the minimum possible number of page faults. Serves as a benchmark for evaluating other algorithms.
- **Disadvantages:** Cannot be implemented in practice (requires future knowledge). Used only for theoretical comparison.

Roll No: _____ **Date:** _____

Program 1: FIFO Page Replacement

Aim: Write a C program to demonstrate FIFO page replacement Algorithm.

Source Code

```c
#include <stdio.h>

int main() {
    int framesCount, pagesCount, i, j, hit = 0, miss = 0;

    printf("Enter frame size: ");
    scanf("%d", &framesCount);

    printf("Enter number of pages: ");
    scanf("%d", &pagesCount);

    int pages[pagesCount];
    printf("Enter the pages:\n");
    for (i = 0; i < pagesCount; i++) {
        scanf("%d", &pages[i]);
    }

    int frames[framesCount];
    for (i = 0; i < framesCount; i++) {
        frames[i] = -1; // initialize frames as empty
    }

    int index = 0; // FIFO pointer

    for (i = 0; i < pagesCount; i++) {
        int found = 0;

        // Check if page already in frames (HIT)
        for (j = 0; j < framesCount; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                hit++;
                break;
            }
        }

        // If not found => MISS (Replace using FIFO)
        if (!found) {
            frames[index] = pages[i];
            index = (index + 1) % framesCount;
            miss++;
        }

        // Print current frame status
        printf("Page %d -> ", pages[i]);
        for (j = 0; j < framesCount; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
```

```
        }

        if (found)
            printf("(HIT)\n");
        else
            printf("(MISS)\n");
    }

    float hitRatio = (float)hit / pagesCount;
    float missRatio = (float)miss / pagesCount;

    printf("\nTotal Hits = %d", hit);
    printf("\nTotal Misses = %d", miss);
    printf("\nHit Ratio = %.2f", hitRatio);
    printf("\nMiss Ratio = %.2f\n", missRatio);

    return 0;
}
```

Output

```
student@cselab:~$ gcc fifo_page.c
student@cselab:~$ ./a.out
Enter frame size: 3
Enter number of pages: 12
Enter the pages:
1 3 2 4 2 3 1 4 2 4 1 3
Page 1 -> 1 - - (MISS)
Page 3 -> 1 3 - (MISS)
Page 2 -> 1 3 2 (MISS)
Page 4 -> 4 3 2 (MISS)
Page 2 -> 4 3 2 (HIT)
Page 3 -> 4 3 2 (HIT)
Page 1 -> 4 1 2 (MISS)
Page 4 -> 4 1 2 (HIT)
Page 2 -> 4 1 2 (HIT)
Page 4 -> 4 1 2 (HIT)
Page 1 -> 4 1 2 (HIT)
Page 3 -> 4 1 3 (MISS)

Total Hits = 6
Total Misses = 6
Hit Ratio = 0.50
Miss Ratio = 0.50
```

Roll No: _____ **Date:** _____

Program 2: LRU Page Replacement

Aim: Write a C program to demonstrate LRU page replacement algorithm.

Source Code

```c
#include <stdio.h>

int main() {
    int framesCount, pagesCount, i, j, k, hit = 0, miss = 0;

    printf("Enter frame size: ");
    scanf("%d", &framesCount);

    printf("Enter number of pages: ");
    scanf("%d", &pagesCount);

    int pages[pagesCount];
    printf("Enter the pages:\n");
    for (i = 0; i < pagesCount; i++) {
        scanf("%d", &pages[i]);
    }

    int frames[framesCount], recent[framesCount];
    for (i = 0; i < framesCount; i++) {
        frames[i] = -1; // initialize frames as empty
        recent[i] = 0;  // keep track of last used time
    }

    int time = 0;
    for (i = 0; i < pagesCount; i++) {
        int found = 0;
        time++;

        // Check if page is already in frames (HIT)
        for (j = 0; j < framesCount; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                hit++;
                recent[j] = time; // update last used time
                break;
            }
        }

        // If MISS
        if (!found) {
            int lruIndex = 0;
            int emptyFound = 0;

            // find an empty frame first
            for (j = 0; j < framesCount; j++) {
                if (frames[j] == -1) {
                    lruIndex = j;
                    emptyFound = 1;
                    break;
                }
```

```
            }

            // if no empty frame, find least recently used
            if (!emptyFound) {
                int min = recent[0];
                lruIndex = 0;
                for (k = 1; k < framesCount; k++) {
                    if (recent[k] < min) {
                        min = recent[k];
                        lruIndex = k;
                    }
                }
            }

            frames[lruIndex] = pages[i];
            recent[lruIndex] = time;
            miss++;
        }

        // Print current frame status
        printf("Page %d -> ", pages[i]);
        for (j = 0; j < framesCount; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        if (found)
            printf("(HIT)\n");
        else
            printf("(MISS)\n");
    }

    float hitRatio = (float)hit / pagesCount;
    float missRatio = (float)miss / pagesCount;

    printf("\nTotal Hits = %d", hit);
    printf("\nTotal Misses = %d", miss);
    printf("\nHit Ratio = %.2f", hitRatio);
    printf("\nMiss Ratio = %.2f\n", missRatio);

    return 0;
}
```

Output

```
student@cselab:~$ gcc lru_page.c
student@cselab:~$ ./a.out
Enter frame size: 3
Enter number of pages: 12
Enter the pages:
1 2 3 4 1 2 5 1 2 3 4 5
Page 1 -> 1 - - (MISS)
```

```
Page 2 -> 1 2 - (MISS)
Page 3 -> 1 2 3 (MISS)
Page 4 -> 4 2 3 (MISS)
Page 1 -> 4 1 3 (MISS)
Page 2 -> 4 1 2 (MISS)
Page 5 -> 5 1 2 (MISS)
Page 1 -> 5 1 2 (HIT)
Page 2 -> 5 1 2 (HIT)
Page 3 -> 3 1 2 (MISS)
Page 4 -> 3 4 2 (MISS)
Page 5 -> 3 4 5 (MISS)

Total Hits = 2
Total Misses = 10
Hit Ratio = 0.17
Miss Ratio = 0.83
```